

Collision Handling

When multiple keys hash to the same position, a collision occurs. Collision handling strategies resolve this by either storing multiple items at one position or finding an alternate position.

Chaining (Separate Chaining)

Idea: Each position in the hash table points to a linked list of all keys that hash to that position.

Operations: - **INSERT(key, value):** compute $h(\text{key})$, prepend to the list at that position; $O(1)$ - **SEARCH(key):** compute $h(\text{key})$, scan the list at that position; $O(1 + L)$ where L is list length - **DELETE(key):** compute $h(\text{key})$, remove from the list at that position; $O(1 + L)$

Performance: - If hash is uniform and load factor $\alpha = \frac{n}{m}$ is small, expected list length is $\frac{n}{m} = \alpha$ - Average time becomes $O(1 + \alpha)$, which is $O(1)$ if α is bounded by a constant

Advantages: - Simple to implement - Delete is straightforward - Can handle many collisions without performance degradation as long as load factor stays small

Disadvantages: - Extra space for linked list pointers - Poor cache locality (linked list scattered in memory)

Linear Probing (Open Addressing)

Idea: If position $i = h(\text{key})$ is occupied, try $i + 1, i + 2, \dots$ until an empty slot is found (wrapping around modulo m).

Operations: - **INSERT(key, value):** compute $h(\text{key})$, probe forward until finding an empty slot - **SEARCH(key):** compute $h(\text{key})$, probe forward while key or occupied markers are seen - **DELETE(key):** mark as deleted (not truly empty, so searches continue past it)

Performance: - Average time is $O(\frac{1}{1-\alpha})$ under uniform hashing - If $\alpha < 0.5$, this is roughly $O(1)$ - As $\alpha \rightarrow 1$, performance degrades rapidly

Advantages: - All data in one array; good cache locality - No extra space for pointers

Disadvantages: - Clustering: if many keys collide initially, they occupy consecutive slots, slowing subsequent probes - Must rebuild table more aggressively to keep α low - Primary clustering: keys hash to the same position, then cluster

together - Secondary clustering: keys hash to different positions but probe to overlapping sequences

Variants: - **Quadratic probing:** try $i, i+1, i+4, i+9, \dots$ to reduce clustering
- **Double hashing:** use two hash functions; probe $i, i+h_2(\text{key}), i+2h_2(\text{key}), \dots$

Comparison

Strategy	Space	Cache	Deletion	Best When
Chaining	$O(n + m)$	Poor	Simple	Load factor can be high; deletions frequent
Linear probing	$O(m)$	Good	Marked slots; wasteful	High-performance systems; $\alpha < 0.5$

When to Use What

- **Chaining:** most robust; good for unpredictable workloads; allows higher load factors
- **Linear probing:** better cache; choose if α is controlled and memory is tight

Related

- [[Hash Tables]]
- [[Hash Functions]]